Atît în viaţa de zi cu zi, cît şi în proiectarea algoritmilor de rezolvare a problemelor, alegerea strategiei potrivite are o importanţă deosebită, atît în ce priveşte durata de rezolvare a problemei, cît şi în ce priveşte calitatea rezultatului care se obţine. Metoda **Greedy** se utilizează în cazul în care se face construirea unei soluţii prin parcurgerea în etape a unor soluţii parţiale. La fiecare pas se alege o soluţie parţială,pe baza căreia se va face căutarea în pasul următor. Metoda pe care o discutăm pleacă de la ideea că, dacă în fiecare pas se alege o soluţie optimă pentru pasul respectiv există, o bună şansă de obţinere a unei soluţii optime în final. Din păcate, o astfel de abordare conduce, eventual, la obţinerea rapidă a unei soluţii, dar pe această cale nu se asigură (pentru orice problemă) optimalitatea globală a soluţiei. De asemenea, există probleme pentru care o astfel de abordare poate să nu conducă la o soluţie, cu toate că aceasta există.

O problemă clasică, pentru care se poate încerca abordarea **Greedy**, este problema **rucsacului** : un rucsac cu capacitate limitată, trebuie să fie umplut cu cele mai utile obiecte dintre cele pe care am vrea să le luăm cu noi într-o excursie. Fiecare obiect este caracterizat prin volum şi utilitate. Într-o formulare particulară, această problemă se poate aplica în situaţia în care dorim să facem o plată pentru care, în afară de cerinţa naturală de a fi corectă, se cere să se utilizeze numărul minim de monezi. Se consideră că există un număr oricît de mare din fiecare fel de monedă. O soluţie de tip **Greedy** presupune că se începe cu moneda cea mai mare şi se consideră numărul maxim de monezi necesare, se continuă apoi în aceeaşi manieră cu suma rămasă de plată. Se consideră deci că alegerea în fiecare pas a numărului minim de monezi care aproximează inferior cel mai bine suma curentă este suficientă pentru obţinerea unei soluţii corecte. Să considerăm, de exemplu, că dorim să plătim suma de 360 de lei, avînd la dispoziţie monezi de 100, 50, 20, 10 şi 3 lei. Conform schiţei de rezolvare anterioare, obţinem evoluţia prezentată în tabelul 8.1.

Tabelul 8.1.

sumă curentă de plată	monede selectate	
	valoare	număr
370	100	3
70	50	1
10	10	1

În cazul general, abordarea **Greedy** poate să fie descrisă prin algoritmul 8.1.

Algoritmul 8.1.

```
{ Iniţializări
  succes <- False
  SeMaiPoateîncerca <- False
  repetă
    dacă se mai poate selecta o soluţie parţială
        optimă atunci
      { SeMaiPoateîncerca <- True
        * selectează o soluţie parţială optimă
        dacă s-a obţinut o soluţie finală atunci
          succes <- True
      }
  pînă cînd succes sau nu SeMaiPoateîncerca
  dacă succes atunci
    * afişează soluţie
  altfel
    * afişează mesaj
}
```

Să aplicăm metoda **Greedy** în cazul în care suma de plată este 365 şi din nou avem la dispoziţie monezi de 100, 50, 20, 10 şi 3 lei. În tabelul 8.2. este prezentată evoluţia în acest caz :

Să aplicăm metoda **Greedy** în cazul în care suma de plată este 365 şi din nou avem la dispoziţie monezi de 100, 50, 20, 10 şi 3 lei. În tabelul 8.2. este prezentată evoluţia în acest caz :

Tabelul 8.2.

sumă curentă de plată	monede selectate	
	valoare	număr
365	100	3
65	50	1
15	10	1
5	3	1
2	...	...

Se observă că nu s-a ajuns la o soluţie (soluţia corectă este 3 x 100 + 1 x 50 + 5 x 3 = 365). Această soluţie ar fi fost găsită dacă în caz de insucces, s-ar fi făcut o revenire, modificînd una dintre selecţiile anterioare. O astfel de revenire înseamnă de fapt altă strategie de rezolvare a problemelor (backtraking). Concluzia exemplului anterior este nu că strategia **Greedy** nu este corectă, ci că ea se poate aplica numai pentru anumite tipuri de probleme.

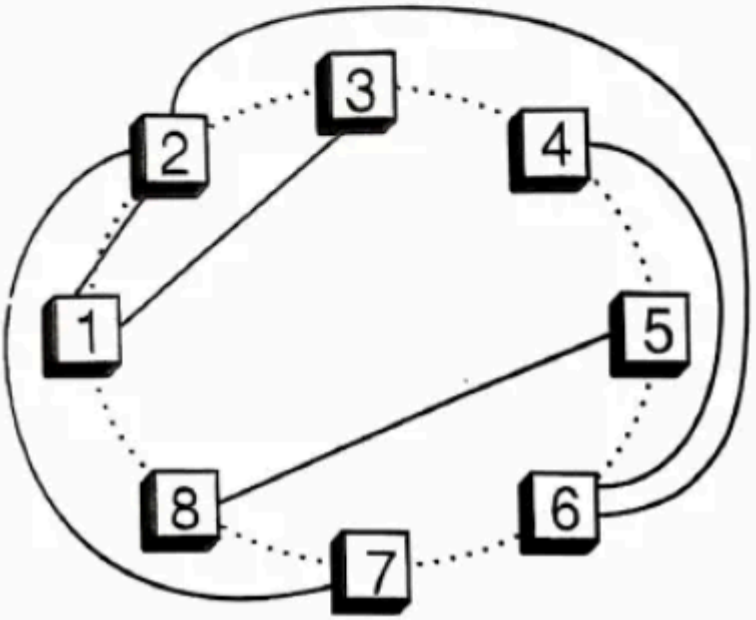
Să considerăm o altă problemă, mai puţin "clasică". În jurul unui oraş există N bazine de alimentare cu apă. Între aceste bazine trebuie să se

sape niște canale. Un bazin poate să fie legat printr-un canal cu mai multe alte bazine. Lista perechilor de bazine ce trebuie să fie legate prin canale este fixată (din considerente exterioare problemei noastre). Un canal se poate construi prin interiorul cercului format de către bazine sau prin exteriorul acestuia. Două canale nu trebuie să se intersecteze.

Să considerăm, de exemplu, situația în care există 8 bazine, numerotate succesiv de la 1 la 8. Lista bazinelor ce trebuie să fie legate între ele este:

1-2, 1-3, 2-7, 4-6, 5-8, 6-2

O soluție posibilă a problemei pentru aceste date este prezentată în figura 8.1.



Pentru a stabili amplasarea canalelor, trebuie să se ţină seama de următoarea restricţie : dacă poziţia unui canal a fost fixată, toate canalele care îl intersectează trebuie să fie amplasate în partea opusă. Este însă posibil ca pentru unele dintre aceste canale să se fi hotărît deja amplasarea, datorită unor condiţionări anterioare. Dacă amplasarea anterioară este în contradicţie cu cea curentă, problema nu are soluţie. Să considerăm de exemplu situaţia în care trebuie să se facă o amplasare de canale conform listei :

(1-4), (3-7), (2-5)

În figura 8.2. se încearcă o soluţie de amplasare.

Se observă că un canal care leagă bazinele 2 şi 5 trebuie să fie construit în partea opusă canalelor 1-4 şi respectiv 3-7. La rîndul lor, aceste două canale nu pot să fie construite de aceeaşi parte (a cercului). Rezultă ca într-o astfel de situaţie problema nu admite soluţie.

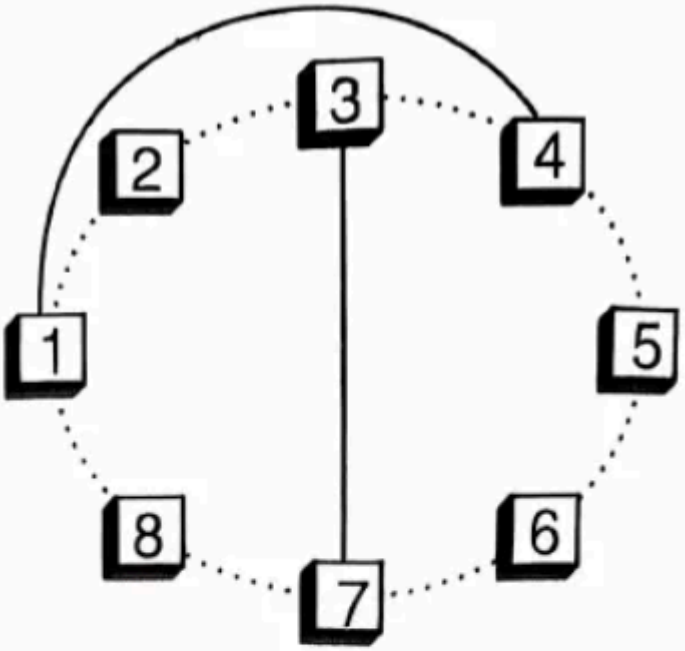


Figura 8.2.

Se poate face observaţia că pe parcursul rezolvării problemei un canal poate să fie în una dintre următoarele stări :

a. nu are o poziţie precizată;

b.are precizată o restricţie de amplasare datorită unor canale care au fost deja amplasate;

c. a fost amplasat.

Trecerea unui canal din starea b în starea c se face după ce s-au explorat toate canalele ce nu au fost încă amplasate şi cu care canalul curent poate să se intersecteze. Dacă vom considera că întotdeauna un canal este specificat printr-o pereche de numere (x, y) cu x mai mic decît y, în figura 8.3. se prezintă situaţiile în care două canale pot să ajungă să se intersecteze şi, deci, trebuie să fie plasate pe poziţii opuse faţă de cerc. Se consideră că există canalul între x1 şi y1 şi se pune problema amplasării unui nou canal între x şi y.

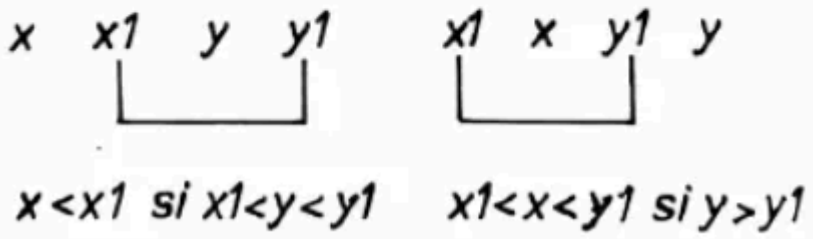


Figura 8.3.

De remarcat faptul că toate relaţiile trebuie să fie stricte (în caz contrar se poate face amplasarea). Într-o abordare de tip **Greedy** se va căuta ca în fiecare pas să fie amplasat cel mai "periculos" canal. Vom considera că cel mai periculos canal este canalul care are numărul maxim de conflicte cu canalele care nu au fost încă amplasate. Considerăm că a fost definit un tip înregistrare numit **CANAL** şi care conţine următoarele cîmpuri:

- două cîmpuri de tip întreg : **Inceput** şi **Sfirsit** ce reprezintă numerele bazinelor între care se construieşte canalul;
- un cîmp de tip enumerare : **PozitieCanal** care poate avea una dintre valorile : **neprecizat, interior** sau **exterior**
- un cîmp de tip logic : **Fixat** care devine **adevărat** după ce canalul a fost amplasat.

Descrierea canalelor este conţinută într-un vector **canal** de înregistrări de tip **CANAL**. Determinarea canalului cel mai periculos se poate face utilizînd funcţiile: **Primul**, **Conditionat**, **NeFixat**, **Contradictie** şi **CanalCuNrMaxContrad**, prezentate în algoritmul 8.2.

Algoritmul 8.2.

```
Contradictie (a, b) este
{  întoarce (( a.Inceput < b.Inceput şi
               a.Sfirsit > b.Inceput şi
               a.Sfirsit < b.Sfirsit) sau
               (a.Sfirsit > b.Sfirsit şi
               a.Inceput > b.Inceput şi
               a.Inceput < b.Sfirsit))
}
```

```
CanalCuNrMaxContrad este
{ dacă exista canale condiţionate neamplasate
  atunci
    k <- Primul canal Condiţionat dar
          neamplasat
  altfel
    k <- Primul canal NeFixat
  PozitieMaxim <- k
  NrMaximContradicţii <- 0
  cît timp (k < N) execută
    { * determină j = numărul maxim de
          contradicţii între canalul k
          şi cele nefixate
      dacă (j > NrMaximContradicţii) atunci
        { PozitieMaxim <- k
          NrMaximContradicţii <- j
        }
      k <- poziţie următorul canal
    }
  întoarce PozitieMaxim
}
```

Funcţia **Primul** caută, dincolo de poziţia transmisă ca prim parametru, primul **canal** care îndeplineşte condiţia verificată de funcţia specificată de al doilea parametru.

Funcţia **Contradicţie** întoarce valoarea **adevărat** dacă numerele de ordine transmise ca argumente sînt cele ale unor canale care nu pot să fie amplasate de aceeaşi parte a cercului.

Funcţia **CanalCuNrMaxContrad** determină canalul nefixat cel mai "periculos" dintre cele cu număr de ordine mai mare decît valoarea

transmisă ca argument. În acest scop este apelată funcţia **Primul**.

Funcţia **AmplaseazaCanal**, prezentată în algoritmul 8.3., este apelată pentru fixarea poziţiei unui canal. Ea analizează toate canalele care trebuie să aibă o poziţie opusă faţă de canalul transmis ca argument. Este suficient ca unul dintre acestea să fie condiţionat şi amplasarea sa să fie similară cu cea a canalului curent, pentru ca problema să nu aibă soluţie.

Algoritmul 8.3.

```
AmplaseazăCanal (curent) este
{ imposibil <- False
  pentru fiecare canal execută
    { dacă nu este canalul curent
            şi nu este un canal fixat
      atunci
        dacă este în conflict cu canalul curent şi
              are aceeaşi amplasare cu canalul
              curent
        atunci
          imposibil <- True;
        altfel
          dacă este în conflict cu canalul curent
          atunci
            *condiţionează amplasarea
              canalului pe poziţia opusă
              canalului curent
    }
  întoarce nu imposibil;
}
```

În algoritmul 8.4. este prezentată rezolvarea, conform metodei **Greedy**, pentru problema propusă, iar programul 8.1. reprezintă implementarea soluţiei propuse.

Algoritmul 8.4.

```
{ succes <- False
  SeMaiPoateIncerca <- False
  *iniţializează i poziţia primului canal tratat
  repetă
    dacă AmplaseazaCanal(i) atunci
      { SeMaiPoateIncerca <- True
        * amplasează canalul i
        i <- CanalCuNrMaxContrad
        dacă i >= N atunci
          succes <- True;
      }
  pînă succes sau nu SeMaiPoateIncerca
  dacă succes
  atunci
    afişează solutie
  altfel
    afisează "nu exista solutie"
}
```